An Exact Right-Recursive Ladder Attack on the Broken SEPAR Implementation

Abstract

We study the *implementation* of SEPAR in `main.c`, not the paper design. The critical implementation error is that the purported 16-bit diffusion layer collapses exactly to

$$L(a, b, c, d) = (b \oplus c,\ b,\ a,\ a \oplus d),$$

which makes every stage byte-triangular and leaves an autonomous 4-bit lane inside every `ENC_Block`. Combined with the matched-context forward/inverse oracle exposed by the word-oriented mode, this yields a Hummingbird-style divide-and-conquer attack. The main contribution of this note is an exact *right-recursive ladder*: after peeling the already known outer right stages, one can inject a candidate block permutation for the next visible stage directly into the reduced inverse oracle. For the true stage key, the current state word cancels from the induced additive differential spectrum exactly. In the state-injected model this exact ladder recovers all segment keys $K_8, \ldots, K_1$. We further show that the 16-bit LFSR contributes no secrecy to same-reset differential attacks and that, once the four-word outer-even core (s_2, s_4, s_6, s_8) is known at two consecutive rounds, the remaining state words (s_1, s_3, s_5, s_7) and the stepped LFSR word are recovered by explicit algebraic formulas.

1 Introduction

The implementation of SEPAR is a cascade of eight 16-bit keyed permutations:

$$F = B_8 \circ B_7 \circ \cdots \circ B_1, \qquad B_i(x) = E_i(x + s_i),$$

where E_i is the stage-local implementation of `ENC_Block`, $s_i \in \mathbb{Z}_{2^{16}}$ is the current state word for stage i, and all additions/subtractions are modulo 2^{16}.

Earlier work on this code base established three structural weaknesses.

(i) **Exact byte triangularity.** The upper output byte of E_i depends only on the upper input byte.

(ii) **Autonomous nibble lane.** The second output nibble of E_i depends only on the second input nibble, giving a true 4-bit keyed permutation channel.

(iii) **Matched-context oracle.** For any chosen prefix, encryption and decryption expose a matched 16-bit permutation pair (F, F^{-1}) for the next word.

The remaining question was whether these primitives can be organized into a genuinely *exact* recursive attack in the style of the classical Hummingbird-1 break, rather than a beam-search engineering attack. This note answers that question affirmatively on the right side of the cascade.

2 The implementation bug

Write a 16-bit word in nibble form as (a, b, c, d) from most significant to least significant nibble. The implementation of `Sep_ROTL16` first performs

$$a \leftarrow a \oplus c, \quad b \leftarrow b \oplus d, \quad c \leftarrow c \oplus b, \quad d \leftarrow d \oplus a,$$

and then xors the result with two rotations.

Theorem 1. *The implemented linear layer satisfies*

$$L(a, b, c, d) = (b \oplus c,\ b,\ a,\ a \oplus d).$$

Proof. This is a direct nibble-wise simplification of the code path in `Sep_ROTL16`. The resulting expression is exact and was exhaustively verified in code. □

Two immediate consequences are central.

Corollary 1. *For each stage i there exists an 8-bit family of functions U_i, L_i such that*

$$E_i(u\|\ell) = U_i(u) \,\|\, L_i(u, \ell),$$

where u is the upper input byte and ℓ is the lower input byte.

Corollary 2. *For each stage i there exists a 4-bit permutation f_i such that the second output nibble of E_i is exactly*

$$\mathrm{nib}_2(E_i(a, b, c, d)) = f_i(b).$$

Thus the full 16-bit block does *not* behave like a well-diffused SP-network. It behaves like a triangular machine whose upper-half branching is driven almost entirely by external carries.

3 Matched-context forward and inverse oracles

Let $P = (p_0, \ldots, p_{m-1})$ be a chosen plaintext prefix and let $C = (c_0, \ldots, c_{m-1})$ be its ciphertext prefix under a fixed IV. Because the encryption and decryption code paths use the *same* post-word state update, the internal state before the next word is the same in

$$\mathrm{Enc}(P\|x) \qquad \text{and} \qquad \mathrm{Dec}(C\|y).$$

Hence we obtain a matched permutation pair

$$F_{P,\mathrm{IV}}(x) = \mathrm{Enc}(P\|x)[m], \qquad G_{C,\mathrm{IV}}(y) = \mathrm{Dec}(C\|y)[m] = F^{-1}_{P,\mathrm{IV}}(y).$$

This is the exact 16-bit handle used throughout the attack. In practice one builds the full codebook of F with 2^{16} chosen encryption queries and the full codebook of G with the corresponding chosen decryption queries.

4 Stage notation

Fix a matched context and suppress the context subscripts. Write

$$F = B_8 \circ B_7 \circ \cdots \circ B_1, \qquad B_i(x) = E_i(x + s_i).$$

The inverse stage is

$$B_i^{-1}(y) = D_i(y) - s_i,$$

where $D_i = E_i^{-1}$ is the stage-local `DEC_Block`.

For $m \in \{1, \ldots, 8\}$ define the right-reduced inverse oracle

$$H_m = G \circ B_8 \circ B_7 \circ \cdots \circ B_{m+1}.$$

When the already peeled outer right stages are correct, this telescopes to

$$H_m = B_1^{-1} \circ B_2^{-1} \circ \cdots \circ B_m^{-1}.$$

5 The exact injected ladder

The core exact identity of this note is obtained by injecting a candidate stage permutation directly into the reduced inverse oracle.

Definition 1 (Injected stage test)**.** *For a candidate key K for stage m, define*

$$J_{m,K}(z) = H_m(E_m(z; K)).$$

Theorem 2 (Exact state cancellation)**.** *Let K_m be the true segment key of stage m. Then*

$$J_{m,K_m}(z) = Q_{m-1}^{-1}(z - s_m),$$

where

$$Q_{m-1} = B_{m-1} \circ B_{m-2} \circ \cdots \circ B_1$$

is the inner $(m-1)$-stage cascade.

Proof. Substitute $K = K_m$:

$$J_{m,K_m}(z) = H_m(E_m(z; K_m)).$$

Since

$$H_m = B_1^{-1} \circ \cdots \circ B_m^{-1}, \qquad B_m^{-1}(y) = D_m(y) - s_m,$$

we obtain

$$B_m^{-1}(E_m(z; K_m)) = z - s_m.$$

Therefore

$$J_{m,K_m}(z) = B_1^{-1} \circ \cdots \circ B_{m-1}^{-1}(z - s_m) = Q_{m-1}^{-1}(z - s_m). \qquad \square$$

The decisive feature is that the current state word s_m appears only as an *input translation.* Hence additive differentials of J_{m,K_m} are translation-invariant:

$$J_{m,K_m}(z + \Delta) - J_{m,K_m}(z) = Q_{m-1}^{-1}(z + \Delta - s_m) - Q_{m-1}^{-1}(z - s_m).$$

The shift by s_m cancels.

5.1 Exact stage score

For a fixed difference set $\mathcal{D} \subset \mathbb{Z}_{2^{16}}^*$, define

$$S_m(K) = \sum_{\Delta \in \mathcal{D}} \max_{\beta \in \mathbb{Z}_{2^{16}}} \left| \{ z \in \mathbb{Z}_{2^{16}} : J_{m,K}(z + \Delta) - J_{m,K}(z) = \beta \} \right|.$$

For the true key K_m, the value of $S_m(K_m)$ reflects the exact additive-differential spectrum of the inner reduced inverse cascade Q_{m-1}^{-1}. Wrong keys generally destroy that structure and score substantially lower.

6 Lane-class reduction

The autonomous second nibble of E_i depends only on a 16-bit abstract signature derived from the two stage words. Concretely, the lane signature of a stage key pair (k_0, k_1) at stage i is

$$\sigma_i(k_0, k_1) = (\mathrm{nib}_2(k_0), \mathrm{nib}_2(k_2), \mathrm{nib}_2(k_1), \mathrm{nib}_2(k_3)),$$

where k_2, k_3 are the stage-local derived words used in `ENC_Block`. This collapses the 2^{32} segment-key space to an average lane class of about 2^{16} candidates. The exact injected score above is then applied only inside the correct lane class.

7 Verified ladder

The file `ladder.py` contains a focused implementation of this exact injected ladder. In the *state-injected* model, where the already peeled outer stage states are given, the ladder recovers all segment keys from right to left.

Target stage	IV used	Exact rank of true key	Comment
K_8	`4703EAC61B442157747A61DDA8FDDDD3`	1	outer injected test
K_7	`4703EAC61B442157747A61DDA8FDDDD3`	1	same context
K_6	`E1A73BD303D52E6811E12AF93264B592`	1	exact shortlist rescore
K_5	`D95A1E433F62724C1FACCB1919637131`	1	exact shortlist rescore
K_4	`4703EAC61B442157747A61DDA8FDDDD3`	1	exact shortlist rescore
K_3	`D95A1E433F62724C1FACCB1919637131`	1	exact shortlist rescore
K_2	`D95A1E433F62724C1FACCB1919637131`	1	exact shortlist rescore
K_1	`D95A1E433F62724C1FACCB1919637131`	1	perfect linear residual

Table 1: Verified exact right-recursive ladder in the state-injected model.

The most striking endpoint is stage 1. Once the outer states are peeled, we have

$$H_1 = B_1^{-1}, \qquad J_{1,K_1}(z) = z - s_1.$$

Hence every additive differential is constant:

$$J_{1,K_1}(z + \Delta) - J_{1,K_1}(z) = \Delta \quad \text{for all } z, \Delta.$$

This was observed in the implementation as a perfect exact score of $6 \cdot 2^{16}$ for the tested set $\mathcal{D} = \{1, 2, 4, 8, 15, 16\}$.

8 The LFSR disappears from same-reset differentials

The Hummingbird-1 attack relies heavily on the fact that the 16-bit LFSR evolves independently from message data. The same is true here. After initialization, the update law in `main.c` is

$$\begin{aligned}
s_2' &= s_2 + v_{12} + v_{56} + s_6, \\
s_3' &= s_3 + v_{23} + v_{34} + s_4 + s_1, \\
s_4' &= s_4 + v_{12} + v_{45} + s_8, \\
s_5' &= s_5 + v_{23} + L', \\
s_6' &= s_6 + v_{12} + v_{45} + s_7, \\
s_7' &= s_7 + v_{23} + v_{67}, \\
s_8' &= s_8 + v_{45}, \\
s_1' &= s_1 + s_5 + v_{34} + 2v_{23} + v_{78},
\end{aligned}$$

where $L' = \mathrm{STEP}(L)$ is the independently updated LFSR word.

Theorem 3 (Exact LFSR cancellation in same-reset pairs)**.** *Consider two encryptions (or two decryptions) started from the same IV and processed for the same number of words. Let*

$\Delta x = x' - x$ *denote additive difference modulo* 2^{16}*. Then the differential state update is*

$$\begin{aligned}
\Delta s_2' &= \Delta s_2 + \Delta v_{12} + \Delta v_{56} + \Delta s_6, \\
\Delta s_3' &= \Delta s_3 + \Delta v_{23} + \Delta v_{34} + \Delta s_4 + \Delta s_1, \\
\Delta s_4' &= \Delta s_4 + \Delta v_{12} + \Delta v_{45} + \Delta s_8, \\
\Delta s_5' &= \Delta s_5 + \Delta v_{23}, \\
\Delta s_6' &= \Delta s_6 + \Delta v_{12} + \Delta v_{45} + \Delta s_7, \\
\Delta s_7' &= \Delta s_7 + \Delta v_{23} + \Delta v_{67}, \\
\Delta s_8' &= \Delta s_8 + \Delta v_{45}, \\
\Delta s_1' &= \Delta s_1 + \Delta s_5 + \Delta v_{34} + 2\Delta v_{23} + \Delta v_{78}.
\end{aligned}$$

In particular, the LFSR contributes no term at all to the same-reset differential recurrence.

Proof. The two executions have the same initial LFSR value because they use the same IV, and the LFSR evolves independently of message words. Hence the same L' is added to both copies of s_5. Subtracting the two update laws therefore cancels the LFSR term exactly. □

This is the exact analogue of the Hummingbird-1 observation that the independent LFSR is not a meaningful barrier to differential cryptanalysis.

9 A four-word exact outer-even core

The previous theorem says that the LFSR disappears in differential space, but we can go further and isolate an even smaller exact state basis than the five-word core used in the earlier draft.

Definition 2 (Outer-even core)**.** *For round t, define the four-word outer-even core*

$$Y_t = (s_{2,t}, s_{4,t}, s_{6,t}, s_{8,t}).$$

Theorem 4 (Exact full-state reconstruction from two outer-even values)**.** *Let p_t be the round-t plaintext word, let $Y_t = (s_2, s_4, s_6, s_8)$, and let $Y_{t+1} = (s_2', s_4', s_6', s_8')$. Then the full round-$t$ state*

$$S_t = (s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$$

is determined exactly by

$$\begin{aligned}
v_{45} &= s_8' - s_8, \\
v_{12} &= s_4' - s_4 - s_8', \\
v_{56} &= s_2' - s_2 - s_6 - v_{12}, \\
s_1 &= D_1(v_{12}) - p_t, \\
s_5 &= D_5(v_{56}) - v_{45}, \\
s_7 &= s_6' - s_6 - v_{12} - v_{45}, \\
v_{23} &= E_2(v_{12} + s_2), \\
v_{34} &= D_4(v_{45}) - s_4, \\
s_3 &= D_3(v_{34}) - v_{23},
\end{aligned}$$

where $D_i = E_i^{-1}$ is the stage-local inverse block.

Proof. The update law gives $v_{45} = s_8' - s_8$ immediately and hence

$$v_{12} = s_4' - s_4 - v_{45} - s_8 = s_4' - s_4 - s_8'.$$

Then

$$v_{56} = s'_2 - s_2 - s_6 - v_{12}.$$

Stage 1 is $v_{12} = E_1(p_t + s_1)$, so $s_1 = D_1(v_{12}) - p_t$. Stage 5 is $v_{56} = E_5(v_{45} + s_5)$, so $s_5 = D_5(v_{56}) - v_{45}$. The s_6 update gives

$$s'_6 = s_6 + v_{12} + v_{45} + s_7,$$

hence $s_7 = s'_6 - s_6 - v_{12} - v_{45}$. Since stage 2 is $v_{23} = E_2(v_{12} + s_2)$, v_{23} is now known exactly. Stage 4 gives $v_{34} = D_4(v_{45}) - s_4$, and stage 3 then gives $s_3 = D_3(v_{34}) - v_{23}$. □

Corollary 3 (The old five-word core is redundant)**.** *From (Y_t, Y_{t+1}, p_t) one also recovers*

$$X_t = (s_{2,t}, s_{4,t}, s_{6,t}, s_{7,t}, s_{8,t}),$$

so the previous five-word core is strictly dominated by the outer-even four-word core.

Proof. The theorem recovers $s_{7,t}$ exactly, while the other four coordinates of X_t are already present in Y_t. □

Corollary 4 (Exact LFSR recovery from two adjacent outer-even pairs)**.** *Given Y_t, Y_{t+1}, Y_{t+2} and plaintext words p_t, p_{t+1}, one recovers the stepped LFSR word exactly:*

$$L_{t+1} = s_{5,t+1} - s_{5,t} - v_{23,t}.$$

Proof. Apply the theorem to (Y_t, Y_{t+1}, p_t) and $(Y_{t+1}, Y_{t+2}, p_{t+1})$ to recover $s_{5,t}$, $s_{5,t+1}$ and $v_{23,t}$ exactly. The update equation

$$s_{5,t+1} = s_{5,t} + v_{23,t} + L_{t+1}$$

then yields the stated formula. □

In fact, among same-index state subsets reused at both rounds, the set $\{2, 4, 6, 8\}$ is the unique size-4 subset from which the full current state is recovered by sequential elimination of the exact update equations. This was verified exhaustively in the companion implementation.

This shows that the independent 16-bit LFSR does not enlarge the true algebraic difficulty of the state problem. Once the outer key ladder is in hand, public state recovery need only target the four-word core Y_t across adjacent contexts; the remaining four state words and the stepped LFSR word are explicit consequences.

10 A public exact state equation

The same ladder identity used for stage-key recovery can also be turned around and used as a *public state test.*

Assume the outer right stage keys $K_{m-1}, K_m, \ldots, K_8$ are known and that the already peeled outer states $s_{m+1}, \ldots, s_8$ are given. Let G be the public inverse next-word permutation for the chosen context and define

$$H_{m-1}^{(\tau)} = G \circ B_8^{(s_8)} \circ \cdots \circ B_{m+1}^{(s_{m+1})} \circ B_m^{(\tau)},$$

where

$$B_i^{(\sigma)}(x) = E_i(x + \sigma).$$

Theorem 5 (Exact public state test)**.** *For the true state word $\tau = s_m$ and the true next visible stage key K_{m-1},*

$$H_{m-1}^{(s_m)}(E_{m-1}(z; K_{m-1})) = Q_{m-2}^{-1}(z - s_{m-1}).$$

Hence the exact additive-differential score of the right-hand side can be used directly as a public test for s_m.

Proof. The already peeled outer stages cancel exactly, and for the true guess $\tau = s_m$ we obtain

$$B_m^{-1}(E_m(z + s_m)) = z.$$

Injecting the next visible key K_{m-1} then reduces the composition to the inner inverse cascade Q_{m-2}^{-1} with only the next state word s_{m-1} left as an input translation. □

Thus state recovery may proceed by the same divide-and-conquer logic as key recovery:

(i) guess the current outer state word s_m;

(ii) peel it publicly into the inverse oracle;

(iii) inject the already known next key K_{m-1};

(iv) select the state guess that maximizes the exact next-stage ladder score.

In the present Python implementation a full 2^{16} exact state scan is still costly, so the script performs a bytewise version of this exact test: it samples all 256 candidates of one byte, exact-rescores the shortlist, and then repeats for the other byte.

Verified public state-byte recoveries

The new `state-byte-scan` mode in `ladder.py` verifies this public exact state equation on the first two right-side states.

- For s_8 on IV `4703EAC61B442157747A61DDA8FDDDD3`, using the public exact K_7 score:
  - low byte: sampled rank 2, exact shortlist rank 1 at the true value 69;
  - upper byte (with low byte fixed): sampled rank 6, exact shortlist rank 1 at the true value $2B$.
- For s_7 on IV `E1A73BD303D52E6811E12AF93264B592`, with the correct outer state $s_8 =$ `1ECC` peeled and using the public exact K_6 score:
  - low byte: sampled rank 1, exact shortlist rank 1 at the true value 02;
  - upper byte (with low byte fixed): sampled shortlist contains the true value, and exact shortlist rank 1 at the true value $D5$.

These experiments are not yet a full public recursive state recovery, but they show that the exact ladder signal remains usable on the state side once the next stage key is known.

11 An exact low-byte state primitive

There is a second exact state observable that is cheaper than the full next-stage ladder score and, unlike the high-byte problem, does not depend on fixing the output translation gauge.

Let F_m be a public forward reduced table at stage m after all outer stages $m+1, \ldots, 8$ have been peeled exactly. For the true stage key K_m, define the peeled table

$$W_m(x) = D_m(F_m(x)).$$

For the true state word $s_m = 256h + \ell$ this has the form

$$W_m(x) = T_{m-1}(x) + 256h + \ell,$$

where T_{m-1} is the remaining inner cascade.

For a fixed row $r \in \{0, \ldots, 255\}$, define

$$\sigma_r(b) = \left|\left\{\mathrm{hi}\big(W_m(256r + \lambda) - b\big) : \lambda \in \{0, \ldots, 255\}\right\}\right|,$$

and the all-row score

$$\Sigma(b) = \sum_{r=0}^{255} \sigma_r(b).$$

Subtracting a candidate low byte b changes the upper byte only through the carry bit from the low byte. Hence $\Sigma(b)$ is an exact deterministic carry-profile score for the unknown low byte ℓ.

Verified exact low-byte recoveries

The new `low-byte-scan` mode in `ladder.py` verifies that, on the tested exact right-recursive contexts, the true low byte is the unique minimizer of $\Sigma(b)$.

stage	IV	true word	exact low-byte rank
8	`4703EAC61B442157747A61DDA8FDDDD3`	`2B69`	1
7	`4703EAC61B442157747A61DDA8FDDDD3`	`C29E`	1
6	`4703EAC61B442157747A61DDA8FDDDD3`	`8E72`	1
4	`D95A1E433F62724C1FACCB1919637131`	`D547`	1
2	`D95A1E433F62724C1FACCB1919637131`	`267C`	1
1	`D95A1E433F62724C1FACCB1919637131`	`599F`	1

Representative scores were:

- stage 8: $\Sigma(\texttt{69}) = 3531$, next best $\Sigma(\texttt{68}) = 3763$;
- stage 7: $\Sigma(\texttt{9E}) = 2413$, next best $\Sigma(\texttt{9D}) = 2657$;
- stage 6: $\Sigma(\texttt{72}) = 1654$, next best $\Sigma(\texttt{71}) = 1900$;
- stage 2: $\Sigma(\texttt{7C}) = 512$, next best $\Sigma(\texttt{7B}) = 768$;
- stage 1: $\Sigma(\texttt{9F}) = 256$, all next-best scores start at 512.

So the current exact public state picture is now asymmetric:

- the low byte is directly exposed by a deterministic all-row carry-support score;
- the remaining part is the state high byte, which can now also be exposed exactly once the next visible stage key is known.

12 An exact high-byte and next-key support primitive

The low-byte primitive above removes the carry ambiguity from the current stage translation. The remaining hidden quantity is the current stage high byte. The key point is that, after the current low byte is removed, the next peeled stage sees only an *upper-byte* translation before its own byte-triangular permutation.

Let F_m again be the forward reduced table at stage m, and let

$$W_m(x) = D_m(F_m(x)).$$

Write the true state as

$$s_m = 256h_m + \ell_m.$$

After recovering ℓ_m exactly, define the low-aligned table

$$\widetilde{W}_m(x) = W_m(x) - \ell_m.$$

For a candidate next key K for stage $m-1$ and a candidate current high byte h, define for each upper-input row $r \in \{0, \ldots, 255\}$ the minimized support

$$\rho_r(K,h) = \min_{b \in \{0,\ldots,255\}} \left| \left\{ \mathrm{hi}\Big(D_K\big(\widetilde{W}_m(256r+\lambda) - 256h\big) - b \Big) : \lambda \in \{0,\ldots,255\} \right\} \right|.$$

Then define the total support score

$$\mathcal{H}_m(K,h) = \sum_{r=0}^{255} \rho_r(K,h).$$

For the true pair (K_{m-1}, h_m), subtracting $256h_m$ removes the remaining translation of the current stage. Applying the correct next-stage inverse yields a byte-triangular reduced table again, and the minimization over b removes the next stage's unknown low-byte translation row by row. Thus $\mathcal{H}_m(K,h)$ is an exact branchless score for the joint pair "current high byte and next stage key."

For stage $m = 2$, the next stage is the first stage, and the same phenomenon becomes a constancy score. After the full current translation is removed, define

$$\mathcal{C}_1(K,h) = \max_{\delta \in \mathbb{Z}_{2^{16}}} \left| \left\{ x \in \mathbb{Z}_{2^{16}} : D_K\big(W_2(x) - (\ell_2 + 256h)\big) - x = \delta \right\} \right|.$$

For the true (K_1, h_2), the peeled stage-1 table differs from the identity by the constant translation s_1, so $\mathcal{C}_1$ is maximized exactly at the true high byte and the true first-stage key.

Verified exact high-byte recoveries

The new `high-byte-scan` mode in `ladder.py` validates this exact high-byte finish on the target instance.

stage	IV	true word	criterion	exact high-byte rank
8	`4703EAC61B442157747A61DDA8FDDDD3`	`2B69`	$\mathcal{H}_8(K_7,h)$	1
7	`4703EAC61B442157747A61DDA8FDDDD3`	`C29E`	$\mathcal{H}_7(K_6,h)$	1
6	`D95A1E433F62724C1FACCB1919637131`	`59DD`	$\mathcal{H}_6(K_5,h)$	1
4	`D95A1E433F62724C1FACCB1919637131`	`D547`	$\mathcal{H}_4(K_3,h)$	1
2	`D95A1E433F62724C1FACCB1919637131`	`267C`	$\mathcal{C}_1(K_1,h)$	1

Representative exact scores were:

- stage 8: the true high byte `2B` scored 136, next best score 174;
- stage 7: the true high byte `C2` scored 100, next best score 119;
- stage 6: the true high byte `59` scored 237, next best score 319;
- stage 4: the true high byte `D5` scored 64, next best score 101;
- stage 2: the true high byte `26` achieved the perfect constancy score $2^{16} = 65536$, while the next best scores were only 468.

Verified joint next-key ranking on sampled lane classes

The same score also ranks the next visible key correctly on sampled same-lane candidate bundles.

- stage $8 \to 7$, IV `4703EAC61B442157747A61DDA8FDDDD3`: on an 8-pair sampled stage-7 lane class, the true $K_7 = (\texttt{8D1E}, \texttt{9DF5})$ ranked 1 and the minimizing high byte was the true value `2B`;
- stage $7 \to 6$, same IV with $s_8 = \texttt{2B69}$ peeled: on an 8-pair sampled stage-6 lane class, the true $K_6 = (\texttt{F44A}, \texttt{233E})$ ranked 1 and the minimizing high byte was the true value `C2`.

So the remaining public state/key problem is now much more structured than before:

- recover the current low byte exactly by the all-row carry/support minimum;
- recover the current high byte and the next key jointly by the exact branchless support score $\mathcal{H}_m$;
- use the stage-1 constancy score $\mathcal{C}_1$ at the bottom of the recursion.

A Hummingbird-style exact one-step lookahead

The next natural step, inspired by the multi-step exact equations in the Hummingbird attacks, is to use the recovered current state word as a *recursive exact consistency check* on the next stage.

For a candidate next key K at stage $m-1$, let

$$\widehat{h}_m(K) = \arg\min_h \mathcal{H}_m(K, h), \qquad \widehat{s}_m(K) = \ell_m + 256\widehat{h}_m(K),$$

where ℓ_m is the exact low byte recovered by the carry/support primitive. Define the candidate next peeled table

$$\widehat{W}_{m-1}^{(K)}(x) = D_K\big(W_m(x) - \widehat{s}_m(K)\big).$$

If $K = K_{m-1}$, then

$$\widehat{W}_{m-1}^{(K_{m-1})}(x) = T_{m-2}(x) + s_{m-1},$$

so the exact low-byte primitive applies again to $\widehat{W}_{m-1}^{(K_{m-1})}$ and must recover the true next low byte ℓ_{m-1}.

This gives a second exact criterion on the same candidate:

(i) minimize the current high-byte support score $\mathcal{H}_m(K, h)$;

(ii) peel with the resulting $\widehat{s}_m(K)$;

(iii) require the next-stage low-byte support score to behave like a genuine reduced stage.

Thus the attack is no longer a one-layer exact test. It is a short exact equation cascade:

$$K_{m-1} \implies \widehat{h}_m(K) \implies \widehat{W}_{m-1}^{(K)} \implies \text{exact next low byte.}$$

Verified recursive exact lookahead

The new `recursive-pair-scan` mode in `ladder.py` validates this one-step exact lookahead on sampled same-lane candidate bundles.

current stage	IV	true next key	true next low	exact pair rank	be
$8 \to 7$	`4703EAC61B442157747A61DDA8FDDDD3`	`8D1E,9DF5`	`9E`	1	
$7 \to 6$	`4703EAC61B442157747A61DDA8FDDDD3`	`F44A,233E`	`72`	1	
$6 \to 5$	`D95A1E433F62724C1FACCB1919637131`	`50C5,12DB`	`DB`	1	
$4 \to 3$	`D95A1E433F62724C1FACCB1919637131`	`02DD,3972`	`62`	1	

Representative measured tuples were:

- stage $8 \to 7$: true pair $(136, \texttt{2B}, 2413, 244)$ versus wrong pairs such as $(171, \texttt{A0}, 3531, 247)$;
- stage $7 \to 6$: true pair $(100, \texttt{C2}, 1654, 246)$ versus wrong pairs such as $(109, \texttt{95}, 2413, 244)$;
- stage $6 \to 5$: true pair $(237, \texttt{59}, 3697, 236)$ versus wrong pairs such as $(316, \texttt{9E}, 5403, 229)$;
- stage $4 \to 3$: true pair $(64, \texttt{D5}, 1024, 254)$ versus wrong pairs such as $(96, \texttt{42}, 2017, 244)$.

Here each tuple is

$$(\text{current high-score},\ \widehat{h}_m,\ \text{next low support score},\ \text{next low gap}).$$

This is the cleanest recursive public filter obtained so far. Wrong candidates do not merely score worse at the current stage; after the candidate peel they fail to produce the exact next-stage low-byte structure and typically collapse to the generic low-byte winner `00`. That is much closer to the exact multi-step logic used in the Hummingbird cryptanalysis than the earlier one-layer public scans.

13 An exact upper-transport law on the reduced inverse chain

The previous sections worked with the right-recursive forward peel. There is a second exact recursion law on the true reduced inverse chain that is even closer to the "prepared suffix" logic in Hummingbird-style attacks.

Let

$$T_i = B_i^{-1} \circ B_{i+1}^{-1} \circ \cdots \circ B_8^{-1}, \qquad i = 1, \ldots, 8,$$

so that

$$T_i(x) = T_{i+1}(D_i(x) - s_i).$$

Write

$$s_i = 256 h_i + \ell_i.$$

For a row index $r \in \{0, \ldots, 255\}$ define the low-corrected upper support of the current reduced table by

$$\mathcal{S}_i(r) = \left\{ \mathrm{hi}\big(T_i(256r + \lambda) + \ell_i\big) : \lambda \in \{0, \ldots, 255\} \right\},$$

and define the raw next-stage upper support by

$$\mathcal{R}_{i+1}(r) = \left\{ \mathrm{hi}\big(T_{i+1}(256r + \lambda)\big) : \lambda \in \{0, \ldots, 255\} \right\}.$$

Let U_i be the upper-byte map of the inverse stage block:

$$U_i(r) = \mathrm{hi}(D_i(256r)).$$

Theorem 6 (Exact upper-transport law)**.** *For every stage $i \in \{1, \ldots, 7\}$, the multiset of low-corrected upper supports of T_i is exactly the stage-i upper map applied to the raw next-stage supports, followed by a global additive shift $-h_i$:*

$$\{\mathcal{S}_i(r) : r \in \mathbb{Z}_{256}\} = \{U_i(\mathcal{R}_{i+1}(r)) - h_i : r \in \mathbb{Z}_{256}\}.$$

Equivalently, the row order may permute, but the full row-support multiset transports exactly through the current inverse stage.

Proof. For fixed r and λ, write

$$D_i(256r + \lambda) = 256u + \mu.$$

Because D_i is byte-triangular, the upper byte u depends only on r. Subtracting $s_i = 256h_i + \ell_i$ gives

$$D_i(256r + \lambda) - s_i = 256(u - h_i - c(\lambda)) + (\mu - \ell_i),$$

where $c(\lambda) \in \{0, 1\}$ is the borrow from subtracting the low byte. Adding back ℓ_i to the *output* of T_i removes the low-byte gauge and leaves only the upper translation by $-h_i$ after transport through T_{i+1}. Since u is row-constant and T_{i+1} contributes only the raw next-stage support set seen at that transported row, the current row support is exactly U_i applied to the corresponding next-stage support, shifted by $-h_i$. Taking multisets over all rows removes the row permutation. □

The theorem has two concrete attack consequences.

(i) Once the next reduced table T_{i+1} is known, the current pair (K_i, s_i) can be tested by *exact* multiset equality rather than by a differential score.

(ii) The unknown current high byte h_i is no longer a nuisance variable; it is exactly the unique shift that makes the transported multiset match.

Verified exact transport validation

The new `transport-validate` mode in `ladder.py` checks this theorem directly on fresh random contexts. On two independent random IVs, every stage 1 through 7 matched exactly:

$$14/14$$

full upper-support multiset matches and

$$14/14$$

canonical multiset matches.

Verified exact pair recovery from the next reduced table

The stronger point is that the transport law is selective enough to identify the true current stage pair on sampled same-lane bundles.

stage	IV	true pair	true state	sampled bundle	exact transport
7	`4703EAC61B442157747A61DDA8FDDDD3`	`8D1E,9DF5`	`C29E`	8 pairs	1
6	`E1A73BD303D52E6811E12AF93264B592`	`F44A,233E`	`CA2F`	8 pairs	1
5	`D95A1E433F62724C1FACCB1919637131`	`50C5,12DB`	`A1DB`	8 pairs	1
1	`D95A1E433F62724C1FACCB1919637131`	`E8B9,B733`	`599F`	8 pairs	1

In each checked bundle, the true pair was the *only* candidate with a full multiset match, and the recovered shift was the true value $-h_i \bmod 256$.

So there are now two exact suffix mechanisms available after the bootstrap:

- the injected-differential ladder of Sections 5–8;
- the reduced-inverse transport law of this section.

Both say the same thing in different coordinates: once the outer gauge is fixed, the remainder of the recursion is exact algebra rather than residual brute force.

14 An exact related-state trail

The previous sections attacked a single matched context. We now describe a second, independent exact algebraic primitive that is much closer in spirit to the original Hummingbird-1 cryptanalysis: an exact related-state trail that converts a chosen-IV relation into deterministic one-word behaviour.

Theorem 7 (Exact one-word related-state family)**.** *Let two online SEPAR states differ only in the first word:*

$$\Delta(s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8) = (\delta, 0, 0, 0, 0, 0, 0, 0).$$

Then:

(i) in encryption, if the plaintext difference is $\Delta p = -\delta$, the ciphertext difference is exactly 0 and the post-round state difference is

$$(\delta, 0, \delta, 0, 0, 0, 0, 0);$$

(ii) in decryption, if the ciphertext difference is 0, the plaintext difference is exactly $\Delta p = -\delta$ and the post-round state difference is again

$$(\delta, 0, \delta, 0, 0, 0, 0, 0).$$

Proof. For encryption, the first stage input is $p + s_1$. Under the related pair

$$(p, s_1) \mapsto (p - \delta, s_1 + \delta),$$

this quantity is unchanged, so the stage-1 output v_{12} is identical in both executions. Since $s_2, \ldots, s_8$ are equal initially, every later stage input and every later stage output is also identical, so the ciphertext words are equal. Substituting $\Delta v_{12} = \cdots = \Delta v_{78} = 0$ into the update law gives

$$\Delta s_1' = \delta, \qquad \Delta s_3' = \delta,$$

and every other post-round state difference is 0.

For decryption, the same-ciphertext condition implies that the inverse stages $8, 7, \ldots, 2$ all receive identical inputs because $s_2, \ldots, s_8$ are equal. Hence $v_{78}, v_{67}, \ldots, v_{12}$ are identical in both executions. The only difference appears at the final subtraction

$$p = D_1(v_{12}) - s_1,$$

which immediately gives $\Delta p = -\delta$. The post-round state difference follows from the same update substitution as above. □

This family was validated exhaustively over random states in `ladder.py` for several values of δ, including $\delta =$ `8000` and $\delta =$ `0001`.

Exact second-word exposure

The one-word trail above does not iterate unchanged, but it produces a *deterministic* next-round state:

$$\Delta S_1 = (\delta, 0, \delta, 0, 0, 0, 0, 0).$$

That already exposes a pure inner-stage differential on the next word.

Encryption. Run a second encryption word under the same plaintext difference $\Delta p = -\delta$. Then

$$\Delta(p + s_1) = 0, \qquad \Delta(v_{12} + s_2) = 0,$$

so

$$\Delta v_{12} = 0, \qquad \Delta v_{23} = 0.$$

The first nonzero difference therefore appears at stage 3:

$$\Delta(v_{23} + s_3) = \delta.$$

Thus the second word exposes an exact stage-3 block differential

$$v'_{34} - v_{34} = E_3(x + \delta) - E_3(x), \qquad x = v_{23} + s_3.$$

Decryption. Run a second decryption word under the same ciphertext in both instances. Since only s_1 and s_3 differ,

$$\Delta v_{78} = \Delta v_{67} = \Delta v_{56} = \Delta v_{45} = \Delta v_{34} = 0,$$

and the first nonzero difference appears at stage 3 inverse:

$$v'_{23} - v_{23} = -\delta.$$

Hence the next stage-2 inverse input is known exactly:

$$\Delta(v_{23}) = -\delta, \qquad v'_{12} - v_{12} = D_2(x - \delta) - D_2(x), \qquad x = v_{23}.$$

So once a suitable chosen-IV pair has been found, the *second* word gives an exact single-stage differential exposure: stage 3 on the encryption side and stage 2 on the decryption side.

Current status. The exact trail itself is solved and implemented. The remaining open step is the Hummingbird-style chosen-IV pair search: one needs two IVs whose post-initialization states satisfy the required one-word relation

$$\Delta S_0 = (\delta, 0, 0, 0, 0, 0, 0, 0).$$

The companion script now includes a public fingerprint search harness for such pairs, but a practical pair-finding algorithm is still open.

15 Complexity in the injected-state model

For one matched context, the data cost is the cost of building the next-word codebook:

$$2^{16}$$

chosen words for the forward table and the same order of work for the inverse table if both are built explicitly.

Offline, each exact stage scan costs

$$|\mathcal{C}_m| \cdot |\mathcal{D}| \cdot 2^{16}$$

block evaluations, where $\mathcal{C}_m$ is the scanned lane class for stage m. In the present implementation the lane class is sampled first and then the best shortlist is exact-rescored. The empirical takeaway is not the precise constant, but the stronger structural statement:

> Once the outer right states are provided, the broken implementation admits an exact right-recursive ladder that recovers $K_8, \ldots, K_1$ stage by stage.

16 Toward full public key recovery

The ladder above removes the stage-key problem from the list of open issues. The remaining obstacle is public recovery of the current outer state word s_m at each step.

The exact recursion suggests the right public architecture:

(1) recover the current outer right key/state pair (K_m, s_m);

(2) form H_m publicly;

(3) use the injected exact score to recover K_{m-1} without depending on s_{m-1};

(4) recurse.

The new state algebra refines this picture further: one does not need to recover the full

$$(s_1, \ldots, s_8, L)$$

at each stage. It is enough to recover the four-word outer-even core $Y_t = (s_2, s_4, s_6, s_8)$ on adjacent contexts, because (s_1, s_3, s_5, s_7) and the stepped LFSR word then follow by exact formulas.

The current state of the attack is therefore:

- **Solved exactly:** right-recursive stage-key recovery under injected outer states;
- **Reduced exactly:** the LFSR/state problem collapses to recovery of the four-word outer-even core on adjacent contexts;
- **Solved publicly at the byte level:** the current state low byte is exposed exactly by the all-row carry-support minimum, and the remaining high byte is exposed exactly by $\mathcal{H}_m$ or $\mathcal{C}_1$ once the next visible key candidate is supplied;
- **Partially solved publicly at the key/state level:** on sampled same-lane bundles, $\mathcal{H}_m$ ranks the true next key first while returning the true current high byte;
- **Still open:** turning that exact byte/key algebra into a fully public end-to-end recursion without truth-guided lane-class injection.

17 Reproducing the ladder

The companion script `ladder.py` provides the following core modes.

- `demo`: runs the verified ladder on known good contexts;
- `scan`: evaluates a single stage and IV;

- `search`: searches random IVs for contexts where the sampled injected score is sharp.
- `outer4-demo`: demonstrates exact full-state and LFSR reconstruction from two adjacent outer-even pairs.
- `outer4-validate`: validates the exact same-reset differential recurrence and the outer-even reconstruction formulas over random contexts.
- `core-demo`: retains the earlier five-word-core reconstruction for comparison.
- `core-validate`: retains validation of the earlier five-word-core formulas.
- `trail-validate`: validates the exact related-state family and the exact second-word stage exposures described above.
- `iv-pair-search`: performs a public chosen-IV fingerprint search for candidate related-IV pairs compatible with the exact trail.
- `state-byte-scan`: demonstrates the public exact state-byte test described above.
- `low-byte-scan`: demonstrates the exact all-row carry/support recovery of the current state low byte.
- `high-byte-scan`: demonstrates the exact branchless support / constancy recovery of the current state high byte and, on sampled bundles, the next visible key.
- `recursive-pair-scan`: adds a one-step exact lookahead and ranks the next visible key by whether the resulting candidate peel also yields the correct next-stage low-byte structure.
- `transport-validate`: validates the exact upper-transport law across stages 1 through 7 on fresh random contexts.
- `transport-pair-scan`: uses the true next reduced table to show that the transport law identifies the current stage pair and the current high-byte shift exactly on sampled same-lane bundles.

Typical commands are:

```
python ladder.py --mode scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --target-right 8 --class-limit 64 --sample-step 2048 --sample-count 32 \
  --exact-top 2

python ladder.py --mode scan --iv D95A1E433F62724C1FACCB1919637131 \
  --target-right 1 --class-limit 64 --sample-step 2048 --sample-count 32 \
  --exact-top 8

python ladder.py --mode outer4-demo --iv D95A1E433F62724C1FACCB1919637131 \
  --words 1234,5678

python ladder.py --mode outer4-validate --samples 64 --rounds 5 --seed 11

python ladder.py --mode trail-validate --delta 8000 --samples 128 --seed 5

python ladder.py --mode iv-pair-search --delta 8000 --trials 4096 \
  --probe-words 0000,1111 --truth-check
```

```
python ladder.py --mode state-byte-scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --scan-stage 8 --scan-part low --sample-step 257 --sample-count 256 \
  --exact-top 4 --diffs 0001,0002,0004,0008

python ladder.py --mode state-byte-scan --iv E1A73BD303D52E6811E12AF93264B592 \
  --scan-stage 7 --scan-part low --known-right-states 8=1ECC \
  --sample-step 1024 --sample-count 64 --exact-top 4 \
  --diffs 0001,0002,0004,0008

python ladder.py --mode low-byte-scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --target-right 8 --top 8

python ladder.py --mode high-byte-scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --target-right 8 --candidate-next-pair 8D1E,9DF5 --row-step 16 --top 8

python ladder.py --mode high-byte-scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --target-right 7 --known-right-states 8=2B69 --class-limit 8 \
  --class-seed 1 --row-step 16 --top 8

python ladder.py --mode recursive-pair-scan --iv 4703EAC61B442157747A61DDA8FDDDD3 \
  --target-right 8 --class-limit 8 --class-seed 1 --row-step 16 --top 8

python ladder.py --mode transport-validate --samples 2 --seed 11

python ladder.py --mode transport-pair-scan --target-right 7 --class-limit 8 \
  --class-seed 1 --top 8
```

18 Conclusion

The broken SEPAR implementation yields to an exact Hummingbird-style divide-and-conquer attack on the right side. The essential observation is not merely that differentials are "strong," but that after the outer right peel one may inject a candidate block permutation directly into the reduced inverse oracle. For the true key, the current stage state cancels *algebraically* from the additive differential spectrum. This gives a genuine exact recursive ladder.

The remaining public problem is also narrower than it first appears. The 16-bit LFSR disappears exactly from same-reset differential recurrences, the current state low byte is exposed by a deterministic row-support minimum, the current state high byte is exposed by an exact branchless next-stage support score once a next-key candidate is supplied, and the reduced inverse chain now satisfies an exact upper-transport law that makes the post-bootstrap suffix deterministic. Moreover, the full round state is determined by two consecutive values of the four-word outer-even core (s_2, s_4, s_6, s_8). Thus the unresolved part of the attack is no longer "defeat the LFSR" or "solve the inner recursion," but rather "find a fully public outer bootstrap that fixes the gauge and exposes the first exact reduced chain." That is a substantially smaller and cleaner target for the next pass.